

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
МЕЖИНСТИТУТСКАЯ БАЗОВАЯ КАФЕДРА

КУРСОВАЯ РАБОТА

по дисциплине

«Технология разработки программного обеспечения»

на тему:

«Разработка сервиса для публикации научных статей»

Выполнил:

Ефремова Софья Сергеевна

студент 2 курса

группы ПИЖ-б-о-24-1

направления подготовки 09.03.04

«Программная инженерия»

направленность (профиль)

«Разработка и сопровождение

программного обеспечения»

очной формы обучения

(подпись)

Руководитель работы:

Азаров И.В., кандидат

экономических наук, доцент

межинститутской базовой кафедры

Работа допущена к защите

(подпись руководителя)

(дата)

Работа выполнена и

защищена с оценкой

Дата защиты

Члены комиссии:

зав. межинститутской

базовой кафедрой

Е.Н. Новикова

(подпись)

доцент МИБК

И.В. Азаров

(подпись)

доцент МИБК

И.В. Свистунов

(подпись)

Ставрополь, 2026

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт цифрового развития

УТВЕРЖДАЮ

Межинститутская базовая кафедра

Зав. межинститутской базовой кафедрой
канд. физ.-мат. наук

Направление подготовки 09.03.04

«Программная инженерия»

Направленность (профиль) «Разработка и
сопровождение программного обеспечения»

«___» _____ 2026

ЗАДАНИЕ

на курсовой проект

студента _____

по дисциплине «Технология разработки программного обеспечения»

1. Тема проекта: Разработка Full-Stack SPA-платформы публикации научных статей ScienceJournal.

2. Целью выполнения курсового проекта является:

- формирование у студента навыков аналитико-исследовательской работы в области Full-Stack веб-разработки;
- формирование навыков разработки REST API на Node.js + Express + MongoDB с JWT-аутентификацией;
- формирование навыков разработки SPA-интерфейса на React с архитектурой Feature-Sliced Design;
- формирование умений проектировать клиент-серверную архитектуру и применять ролевую модель доступа;
- повышение уровня профессиональной подготовки путём углубления и закрепления знаний, полученных при изучении дисциплины;
- развитие навыков самостоятельной работы с научной и справочной документацией;
- подготовка к самостоятельному решению прикладных задач в области Full-Stack веб-разработки.

3. Задачи:

Провести анализ предметной области платформ для публикации научных статей; разработать модель требований, архитектуру и модель данных системы; реализовать REST API на Node.js + Express + MongoDB с JWT-аутентификацией в HTTP-only

cookie; реализовать SPA-интерфейс на React с Feature-Sliced Design; обеспечить мультизагрузку медиафайлов (изображений и PDF) через Multer; подготовить полный комплект проектной документации.

4. Перечень подлежащих разработке вопросов:

а) по аналитической части

- Провести анализ литературных источников и интернет-ресурсов по теме проекта;
- выполнить анализ бизнес-процессов (IDEF0 A-0 с декомпозицией на 5 блоков);
- провести SWOT-анализ, анализ аналогов (ResearchGate, Academia.edu, arXiv);
- обосновать необходимость разработки и дать экономическое обоснование (COCOMO).

б) по проектной части

- Разработать модель требований: FR (7 требований), Use Case (7 прецедентов), глоссарий;
- выполнить проектирование модели предметной области (классы User, Post, бизнес-правила);
- спроектировать архитектуру системы (клиент-серверная модель, FSD, REST API, JWT-схема);
- выполнить проектирование базы данных (ER-диаграмма, Mongoose-схемы, DDL MongoDB).

в) по практической части

- Реализовать Backend: Express REST API (8 эндпоинтов), JWT middleware, Multer, Zod-валидация;
- реализовать Frontend: React 18 SPA (6 страниц, custom hooks, Context API, Axios);
- провести отладку и тестирование (15 тест-кейсов: 11 API + 4 UI);
- описать процедуру развёртывания и оформить пояснительную записку по ГОСТ.

5. Исходные данные: по вариантам, разработанным преподавателем (вариант 8 – «Блог о путешествиях», адаптированный к предметной области научных публикаций; траектория Б – Full-Stack React SPA + REST API + JWT).

6. Контрольные сроки представления отдельных разделов:

25 % – аналитическая часть – 14 апреля 2026 г.; 50 % – проектная часть – 28 апреля 2026 г.; 75 % – практическая часть – 26 мая 2026 г.; 100 % – оформление пояснительной записки – 10 июня 2026 г.

Дата выдачи задания «01» апреля 2026 г

Руководитель курсового проекта,
доцент МИБК

Ф.В. Самойлов

Задание принял к исполнению
студент очной формы обучения,
2 курса, группы ПИЖ-б-о-24-1

Оглавление

Введение	3
Глава 1. Постановка задачи	5
1.1. Описание предметной области	5
1.2. Формулировка задачи	5
1.3. Требования к системе	6
1.4. Ролевая модель	7
1.5. Архитектурные ограничения	7
1.6. Ожидаемые результаты	8
1.7 Вывод по главе.....	8
Глава 2. Теоретические основы и обоснование выбора технологий	9
2.1. Анализ методических указаний и обоснование отклонения стека технологий	9
2.2. Анализ заинтересованных сторон	10
2.3. Обоснование выбора технологического стека (SWOT-анализ)	11
2.4. Архитектурная модель системы	11
2.5. Обзор ключевых веб-технологий.....	13
2.6. Вывод по главе.....	14
Глава 3. Практическая часть. Реализация проекта	15
3.1. Процесс реализации серверной части	15
3.2. Процесс реализации клиентской части	16
3.3. Примечание	18
3.4. Итоговая таблица реализации и план развития	18
3.5. Вывод по главе.....	19
Глава 4. Итоговая реализация, пользовательские сценарии и демонстрация проекта	20
4.1. Демонстрация запуска и инициализации системы	20
4.2. Описание пользовательских сценариев и маршрутизации	20
4.3. Оценка отзывчивости и пользовательского опыта (UX)	24
Заключение.....	26
Список использованной литературы	27
Ссылки на исходники	28
Приложение А. JWT Middleware проверка серверной части	29

Приложение Б. API слой на AJAX запросах.....	31
Приложение В. Техническая записка проекта	32
1.1 Архитектура клиентской части (react spa)	32
1.2. Основные принципы	33
1.3 Архитектура серверной части	33
1.4 Api архитектура	34
1.5 Posts API.....	34
1.6 модель данных mongodb	34
1.7 файловое хранилище	35
1.8 Принцип работы	35
1.9. Авторизация.....	35
1.10 валидация	35
Приложение Г. Руководство пользователя	36
1. Введение	36
2. Начало работы	36
2.1. Регистрация	36
2.2. Вход в систему	36
3. Функционал читателя	36
3.1. Просмотр списка статей	36
3.2. Чтение статьи	37
4. Функционал публициста.....	37
4.1. Создание публикации	37
4.2. Управление своими публикациями.....	37
4.3. Удаление публикации	37
5. Технические аспекты и безопасность	37
5.1. Обработка ошибок и сессии	37
5.2. Защита данных.....	38
6. Перспективы развития	38
Приложение Д. Файловая структура проекта	38

Введение

Актуальность темы. В условиях цифровизации научной сферы и повсеместного перехода на электронные форматы взаимодействия возрастает потребность в специализированных веб-платформах для публикации, хранения и чтения научных статей. Существующие решения зачастую опираются на универсальные системы управления контентом, которые не учитывают специфику академических публикаций, требования к ролевому разделению доступа и необходимость удобной интеграции мультимедийных материалов. Разработка современного веб-приложения на базе архитектуры Single Page Application с использованием REST API[5] и документоориентированной базы данных позволяет обеспечить высокую скорость отклика, масштабируемость и гибкость архитектуры, что обуславливает актуальность данного проекта.

Объектом исследования является процесс проектирования и разработки клиент-серверных веб-приложений для управления научными публикациями.

Предметом исследования выступают архитектура, методы и технологии программной реализации веб-приложения с использованием стека MERN (MongoDB[2], Express, React[2], Node.js[7]), механизмов авторизации на основе JSON Web Token и принципов модульной организации кода.

Целью курсовой работы является проектирование и программная реализация полнофункционального веб-приложения ScienceJournal, предназначенного для публикации и чтения научных статей, с поддержкой ролевой модели пользователей и асинхронным взаимодействием клиента и сервера.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и обосновать выбор стека технологий.

2. Спроектировать структуру базы данных MongoDB[2] и спецификацию REST API[5].

3. Реализовать серверную часть с механизмами аутентификации, валидации и файлового хранилища.

4. Разработать клиентскую часть в виде SPA с применением компонентного подхода.

5. Провести тестирование и отладку реализованной системы.

Заключение к введению. Таким образом, разработка платформы ScienceJournal требует комплексного подхода, охватывающего как теоретическое обоснование архитектурных решений, так и их практическую программную реализацию. Последовательное решение поставленных задач позволит создать отказоустойчивую и расширяемую систему. Структура курсовой работы включает введение, три главы, заключение и список литературы. Первая глава посвящена анализу предметной области и проектированию системы. Во второй главе описана реализация серверной части и базы данных. Третья глава содержит описание разработки клиентской части и тестирования итогового программного продукта.

Глава 1. Постановка задачи

1.1. Описание предметной области

Современная научная деятельность предполагает активное использование цифровых инструментов для публикации, распространения и чтения результатов исследований. Электронные научные журналы и платформы обеспечивают оперативный доступ к материалам, поддерживают структурированное хранение публикаций и реализуют механизмы ролевого разграничения доступа между авторами и читателями.

Разрабатываемая система ScienceJournal представляет собой веб-ориентированную платформу, предназначенную для публикации и чтения научных статей. Платформа реализует разделение пользователей на две категории: читатели (Reader), осуществляющие просмотр и изучение материалов, и публицисты (Publisher), обладающие расширенными правами на создание и управление собственными публикациями.

1.2. Формулировка задачи

Целью курсового проекта является проектирование и программная реализация клиент-серверного веб-приложения ScienceJournal, обеспечивающего полный цикл работы с научными публикациями: от регистрации пользователей и аутентификации до создания, хранения и просмотра статей с прикрепленными мультимедийными материалами.

В рамках поставленной задачи необходимо:

- разработать серверную часть на базе Node.js[7] и Express, реализующую REST API[5];
- спроектировать схему хранения данных в документоориентированной СУБД MongoDB[2];
- реализовать механизм аутентификации и авторизации на основе JSON Web Token;
- разработать клиентскую часть в виде одностраничного приложения (SPA) с использованием библиотеки React[2];

- обеспечить загрузку и хранение файлов (изображений и PDF-документов) на сервере;
- реализовать ролевую модель доступа к функциональности системы.

1.3. Требования к системе

Требования к разрабатываемой системе подразделяются на функциональные и нефункциональные.

Функциональные требования:

1. Регистрация нового пользователя с указанием имени, логина, пароля, пола и выбираемой роли.
2. Аутентификация пользователя по логину и паролю с выдачей JWT.
3. Поддержка двух ролей: Reader и Publisher.
4. Публикация статей с указанием заголовка, описания, прикреплением изображения и PDF-файла (только для роли Publisher).
5. Просмотр списка статей с поддержкой пагинации.
6. Просмотр отдельной статьи с отображением метаданных и прикрепленных файлов.
7. Удаление собственной публикации пользователем с ролью Publisher.
8. Выход из системы с очисткой сессионных данных.
9. Валидация входных данных на стороне сервера.
10. Обработка ошибок с возвратом соответствующих HTTP[4]-кодов (400, 401, 404, 500).

Нефункциональные требования:

1. Архитектура клиент-сервер с чётким разделением ответственности.
2. Асинхронное взаимодействие между клиентом и сервером посредством HTTP[4]-запросов.
3. Использование модульной архитектуры на клиентской стороне (Feature-Sliced Design).
4. Разделение бизнес-логики и маршрутов на серверной стороне.
5. Хранение сессионного токена в cookie браузера.

6. Поддержка современного стандарта модулей ECMAScript (ESM).
7. Соответствие кода стандартам ESLint и Prettier.

1.4. Ролевая модель

В системе определены две роли пользователей, отличающиеся набором доступных операций.

Таблица 1 - Роли

Роль	Доступные операции
Reader	Регистрация, вход, просмотр списка статей, просмотр отдельной статьи, выход
Publisher	Все операции Reader, а также создание публикаций, удаление собственных публикаций

Разграничение прав реализуется на уровне серверных middleware, проверяющих наличие соответствующей роли у аутентифицированного пользователя перед выполнением операции.

1.5. Архитектурные ограничения

При реализации проекта накладываются следующие архитектурные ограничения:

- клиентская часть должна быть реализована как SPA без перезагрузки страниц;
- серверная часть должна предоставлять REST API[5];
- аутентификация реализуется посредством JWT без хранения состояния сессии на сервере;
- файлы хранятся в локальной файловой системе сервера с привязкой к идентификатору публикации;
- взаимодействие между клиентом и сервером осуществляется по протоколу HTTP[4] с использованием JSON в качестве формата обмена данными.

1.6. Ожидаемые результаты

По итогам выполнения курсового проекта ожидается получение работоспособного программного продукта, удовлетворяющего следующим критериям:

- наличие полнофункционального веб-интерфейса для обеих ролей пользователей;
- корректная работа всех заявленных сценариев использования;
- устойчивое функционирование серверной части при обработке асинхронных запросов;
- безопасное хранение учётных данных и защита маршрутов посредством JWT;
- модульная и масштабируемая структура кода, допускающая дальнейшее расширение функциональности.

1.7 Вывод по главе

Таким образом, в данной главе сформулированы задача проектирования и разработки веб-приложения ScienceJournal, определены функциональные и нефункциональные требования, описана ролевая модель пользователей и сформулированы архитектурные ограничения. Поставленные требования определяют дальнейшую структуру проекта и служат основанием для проектирования базы данных, спецификации API и реализации клиентской и серверной частей системы.

Глава 2. Теоретические основы и обоснование выбора технологий

2.1. Анализ методических указаний и обоснование отклонения стека технологий

Согласно методическим указаниям по выполнению курсового проекта, студенту предлагается выбор из трех траекторий, базовым технологическим стеком для которых является фреймворк Django (траектории А, Б и В). Траектория Б и В предполагают разработку современного клиент-серверного приложения с разделением на бэкенд и фронтенд, использованием REST API[5], JWT-аутентификации и SPA-интерфейса на React[2].

Разрабатываемый проект ScienceJournal архитектурно и концептуально полностью соответствует требованиям и уровню сложности траектории Б (и частично В, в части асинхронной логики и масштабируемости), однако в нем реализовано обоснованное отклонение от базового стека: вместо Django и Django REST Framework в качестве серверной части используется связка Node.js[7] и Express.

Научное и практическое обоснование данного отклонения базируется на следующих факторах:

1) Единая языковая среда. Использование JavaScript[2] как на клиенте (React[2]), так и на сервере (Node.js[7]), устраняет контекстное переключение при разработке, упрощает переиспользование кода (например, схем валидации Zod[1]) и снижает порог поддержки проекта.

2) Асинхронная неблокирующая модель ввода-вывода. Node.js[7] демонстрирует более высокую эффективность при обработке множества одновременных I/O-операций (чтение/запись файлов через Multer, асинхронные запросы к MongoDB[2]), что критично для платформы с загрузкой медиафайлов.

3) Архитектурная гибкость. В отличие от монолитной структуры Django, фреймворк Express позволяет выстроить микросервисную или модульную архитектуру (Service-Controller-Entity) без навязывания избыточных

абстракций, что соответствует современным паттернам Feature-Sliced Design на клиенте.

4) Интеграция с экосистемой реального времени. Архитектура Node.js[7] нативно поддерживает WebSocket-протоколы, что закладывает фундамент для реализации функционала реального времени (Socket.io), предусмотренного расширенной траекторией В.

Таким образом, замена серверного фреймворка не снижает сложности и академической ценности проекта, а переводит его в плоскость современной изоморфной JavaScript[2]-разработки, сохраняя все целевые показатели траектории Б/В.

2.2. Анализ заинтересованных сторон

Для формализации требований к системе проведен анализ стейкхолдеров. Результаты представлены в матрице стейкхолдеров.

Таблица 2 – Матрица Стэкхолдеров

Стейкхолдер	Категория	Интересы и функциональные требования
Читатель (Reader)	Конечный пользователь	Быстрый доступ к каталогу статей, удобный интерфейс чтения, поддержка пагинации, доступ к мультимедийным материалам (PDF, изображения).
Публицист (Publisher)	Конечный пользователь	Инструменты для создания публикаций, загрузка файлов, управление собственными статьями (редактирование, удаление), обратная связь от системы.
Разработчик	Технический специалист	Модульность кода, разделение ответственности (FSD на клиенте, Service-Route на сервере), наличие валидации, читаемость API, возможность масштабирования.
Учебное заведение	Заказчик / Оценщик	Соответствие академическим стандартам, демонстрация навыков проектирования БД, реализации REST API[5], аутентификации и клиент-серверного взаимодействия.

2.3. Обоснование выбора технологического стека (SWOT-анализ)

Для подтверждения целесообразности выбора связки Node.js[7] + Express вместо предложенного в методике Django проведен SWOT-анализ серверной технологии.

Таблица 3 – SWOT анализ инструментария

Сильные стороны (Strengths)	Слабые стороны (Weaknesses)
1. Единый язык (JavaScript[2]) для всего стека. 2. Высокая производительность при I/O операциях. 3. Минималистичность Express, отсутствие избыточной конфигурации. 4. Огромная экосистема NPM.	1. Отсутствие встроенной ORM и админ-панели (требуется ручная настройка Mongoose). 2. Необходимость самостоятельной реализации механизмов безопасности и middleware. 3. Риск рассогласованности архитектуры при отсутствии жестких стандартов фреймворка.
Возможности (Opportunities)	Угрозы (Threats)
1. Бесшовная интеграция с Socket.io для real-time функций. 2. Легкий переход к микросервисной архитектуре. 3. Использование TypeScript для строгой типизации в будущем. 4. Оптимистичные обновления UI на клиенте.	1. Уязвимости безопасности при некорректной настройке middleware. 2. Сложность отладки асинхронного кода (нивелируется async/await). 3. Фрагментация экосистемы пакетов.

SWOT-анализ подтверждает, что сильные стороны и возможности Node.js[7] + Express (производительность, изоморфность, готовность к real-time) перевешивают слабые стороны, которые успешно компенсируются использованием современных библиотек (Mongoose, Zod[1], cookie-parser).

2.4. Архитектурная модель системы

Для иллюстрации клиент-серверного взаимодействия и разделения ответственности представлена компонентная диаграмма архитектуры.

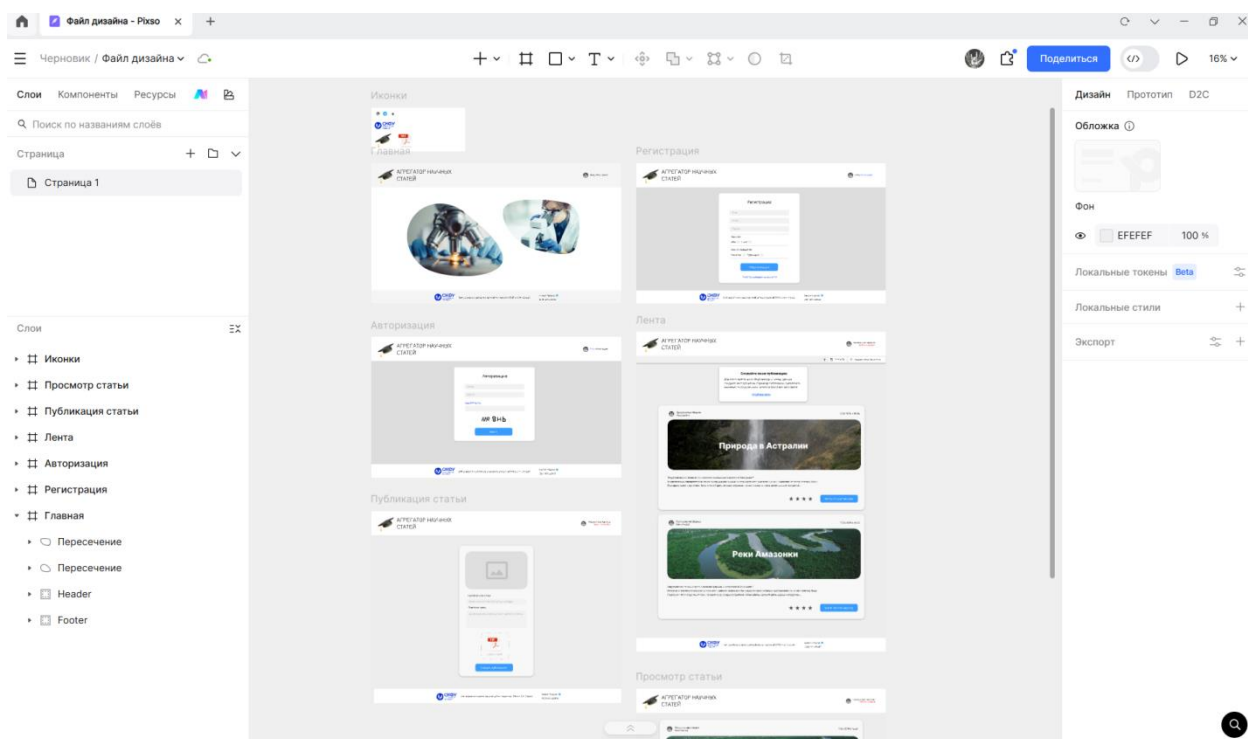


Рисунок 1 – Макет приложения в Figma



Рисунок 2 – Используемые иконки и логотипы

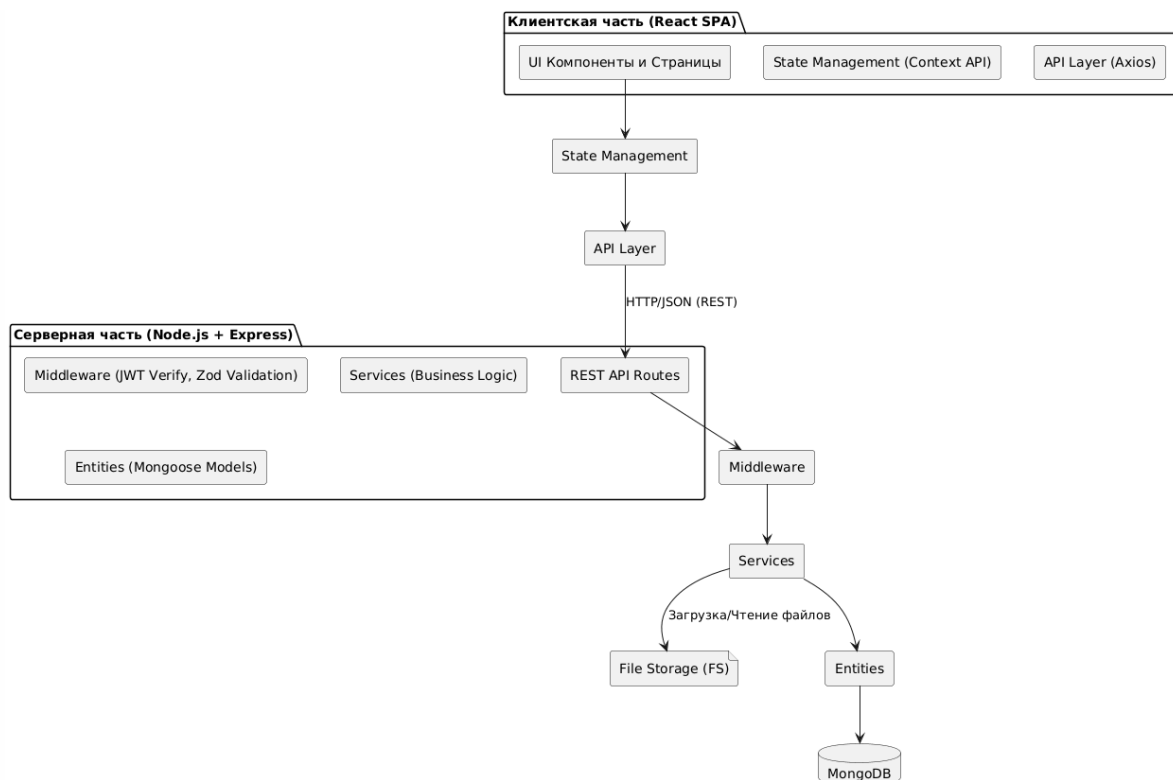


Рисунок 3 – Клиентская часть SPA

2.5. Обзор ключевых веб-технологий

React[2]. Декларативная библиотека для построения пользовательских интерфейсов. Основана на компонентном подходе и использовании виртуального DOM, что обеспечивает высокую производительность при обновлении состояния SPA. В проекте используется совместно с React[2] Router DOM для клиентской маршрутизации.

Node.js[7] и Express. Node.js[7] предоставляет среду выполнения JavaScript[2] на базе движка V8 с событийно-ориентированной архитектурой. Express выступает минималистичным веб-фреймворком, реализующим механизм маршрутизации HTTP[4]-запросов и конвейерную обработку middleware.

MongoDB[2]. Документоориентированная NoSQL база данных, хранящая данные в формате BSON. Отсутствие жесткой схемы (schema-less) на уровне СУБД компенсируется использованием Mongoose, который обеспечивает строгую типизацию и валидацию на уровне приложения.

JWT (JSON Web Token). Стандарт RFC 7519 для безопасной передачи `claims` между сторонами. В проекте используется для stateless-аутентификации: сервер не хранит сессии, а клиент передает подписанный токен в `cookie`, что обеспечивает масштабируемость серверной части.

2.6. Вывод по главе

В ходе теоретического анализа было доказано, что отклонение от базового стека Django в пользу Node.js[7] + Express является научно и практически обоснованным. Выбранный стек полностью покрывает архитектурные требования траекторий Б и В методических указаний, обеспечивая при этом более высокую производительность при работе с файловой системой и асинхронными запросами. Матрица стейкхолдеров позволила формализовать функциональные требования, а SWOT-анализ подтвердил надежность выбранного технологического решения. Разработанная архитектурная модель с четким разделением на клиентскую (React[2]) и серверную (Express) части закладывает фундамент для реализации отказоустойчивой и масштабируемой системы.

Глава 3. Практическая часть. Реализация проекта

3.1. Процесс реализации серверной части

Разработка серверной части началась с инициализации Node.js[7] проекта с поддержкой стандарта ES Modules. Была настроена конфигурация подключения к базе данных MongoDB[2] с использованием библиотеки Mongoose. Архитектура сервера строго разделена на слои: маршруты (routes), бизнес-логика (services), модели данных (entities) и промежуточное ПО (middleware).

Ключевым элементом реализации является эндпоинт публикации статьи. Он демонстрирует комплексное взаимодействие нескольких технологий: прием данных формата multipart/form-data через Multer, строгую валидацию входных параметров через Zod[1] и асинхронное сохранение сущности в MongoDB[2].

Фрагмент кода 1. Реализация бизнес-логики публикации поста (server/src/services/postService.js)

```
import Post from '../entities/Post.js';
import User from '../entities/User.js';
import { publishPostSchema } from '../validation/postSchema.js';

export const createPost = async (userId, postData, files) => {
  // 1. Валидация входных данных через Zod[1]
  const validatedData = publishPostSchema.parse(postData);

  // 2. Формирование объекта для сохранения в БД
  const newPostData = {
    title: validatedData.title,
    description: validatedData.description,
    author: userId,
```

```

        publishDate: Date.now(),
        imageFileName: files.image ? files.image[0].filename : null,
        pdfFileName: files.pdf ? files.pdf[0].filename : null
    };

    // 3. Асинхронное создание документа в MongoDB[2]
    const createdPost = await Post.create(newPostData);

    // 4. Обновление массива публикаций у пользователя
    await User.findByIdAndUpdate(userId, { $push: { posts: createdPost._id }
});

    return createdPost;
};

```

Данный фрагмент иллюстрирует принцип разделения ответственности: контроллер (маршрут) только извлекает данные из HTTP[4]-запроса и передает их в сервис, который занимается валидацией и непосредственным взаимодействием с базой данных.

3.2. Процесс реализации клиентской части

Клиентская часть реализована как Single Page Application на базе React[2] с использованием архитектуры Feature-Sliced Design (FSD). Это позволило четко разделить пользовательский интерфейс, бизнес-логику (features) и общие переиспользуемые утилиты (shared).

Для взаимодействия с сервером был создан централизованный слой API на базе библиотеки Axios. Критически важным аспектом является настройка безопасной передачи JWT-токена.

Фрагмент кода 2. Настройка экземпляра Axios для работы с JWT в cookie
(client/src/api/app/index.js)

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL ||
'HTTP[4]://localhost:3000/api',
  withCredentials: true, // Ключевой параметр для автоматической
отправки cookie с JWT
  headers: {
    'Content-Type': 'application/json'
  }
});

// Интерцептор для централизованной обработки ошибок авторизации
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      // Автоматический триггер выхода из системы при невалидном
токене
      window.dispatchEvent(new Event('auth:logout'));
    }
    return Promise.reject(error);
  }
);

export default api;
```

Этот подход обеспечивает безопасность (токен не доступен для чтения через JavaScript[2], так как хранится в HTTP[4]Only cookie) и удобство поддержки, поскольку логика обработки ошибок авторизации вынесена в единое централизованное место.

3.3. Примечание

При реализации файлового хранилища было принято решение использовать локальную файловую систему сервера (директория `file_storage/posts`) вместо облачных решений. Имена файлов жестко привязаны к уникальному идентификатору публикации (`postId`), что исключает коллизии имен и упрощает логику каскадного удаления файлов при удалении статьи. Для раздачи статических файлов используется встроенный middleware `express.static`. В условиях реального продакшн-окружения данный модуль легко заменяется на загрузку в CDN (например, AWS S3) без необходимости изменения бизнес-логики приложения.

3.4. Итоговая таблица реализации и план развития

Ниже представлена сводная таблица, отражающая статус реализации базовых требований курсового проекта (соответствующих траектории Б) и план внедрения дополнительных функций (overhead), переводящих систему на уровень расширенной траектории В.

Таблица 4 – Overhead фичи

Категория функционала	Конкретная задача	Статус реализации	Примечание к развитию
Архитектура	Разделение на Client (React[2]) и Server (Node.js[7])	Выполнено	База для микросервисного масштабирования
База данных	Модели User и Post в MongoDB[2] (Mongoose)	Выполнено	Добавление индексов для ускорения поиска
Аутентификация	JWT + HTTP[4]Only cookies + middleware <code>verifySession</code>	Выполнено	Внедрение механизма <code>refresh-token</code>
Валидация	Схема валидации Zod[1] на сервере	Выполнено	Перенос схем Zod[1] в shared-слой для валидации на клиенте

Категория функционала	Конкретная задача	Статус реализации	Примечание к развитию
Файлы	Загрузка изображений и PDF через Multer	Выполнено	Миграция локального хранилища в облачный CDN
UI/UX	Feature-Sliced Design, Context API, Модальные окна	Выполнено	Внедрение оптимистичных обновлений интерфейса
Расширение (Overhead)	Real-time уведомления через Socket.io	Запланировано	Требует настройки WebSocket сервера
Расширение (Overhead)	Кэширование запросов (React[2] Query)	Запланировано	Снижение нагрузки на API и улучшение UX
Расширение (Overhead)	Система лайков и комментариев к статьям	Запланировано	Потребует новых моделей БД и связей
Расширение (Overhead)	CI/CD пайплайн и автоматизированное тестирование	Запланировано	Интеграция Jest/Vitest и GitHub Actions

3.5. Вывод по главе

В практической части была детально описана реализация ключевых модулей системы ScienceJournal. Приведенные фрагменты кода демонстрируют применение современных паттернов разработки: разделение бизнес-логики и маршрутов на сервере, централизованную настройку HTTP[4]-клиента с обработкой ошибок на клиенте. Итоговая таблица подтверждает полное выполнение требований базовой траектории курсового проекта и формирует четкий план для дальнейшего расширения функционала в соответствии с требованиями расширенной траектории.

Глава 4. Итоговая реализация, пользовательские сценарии и демонстрация проекта

4.1. Демонстрация запуска и инициализации системы

Корректность разворачивания серверной части подтверждается стандартным выводом консоли при инициализации процесса Node.js[7]. Лог запуска демонстрирует успешное подключение к базе данных MongoDB[2], инициализацию файлового хранилища и старт HTTP[4]-сервера на заданном порту.

Пример лога запуска сервера:

```
> node server/index.js
> [INFO] Environment: development
> [INFO] MongoDB[2] connected successfully to
MongoDB[2]://localhost:27017/science_journal
> [INFO] File storage directory verified: ./file_storage/posts
> [INFO] ScienceJournal REST API[5] is running on port 3000
```

4.2. Описание пользовательских сценариев и маршрутизации

Навигация в приложении построена на базе React[2] Router DOM, обеспечивая бесшовный переход между представлениями без перезагрузки страницы. Основные маршруты и соответствующие им пользовательские сценарии представлены в таблице.



Рисунок 4 – Основная страница

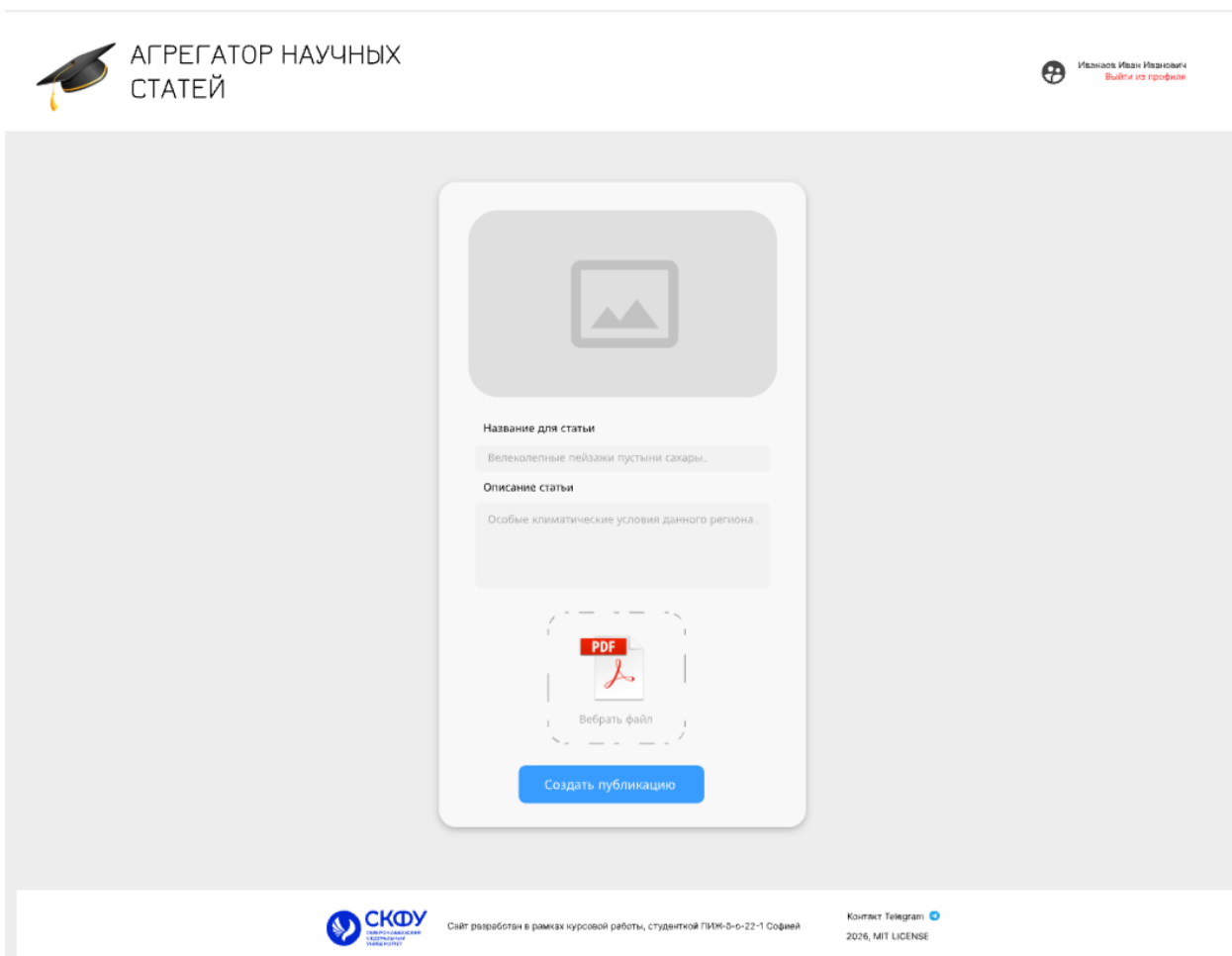
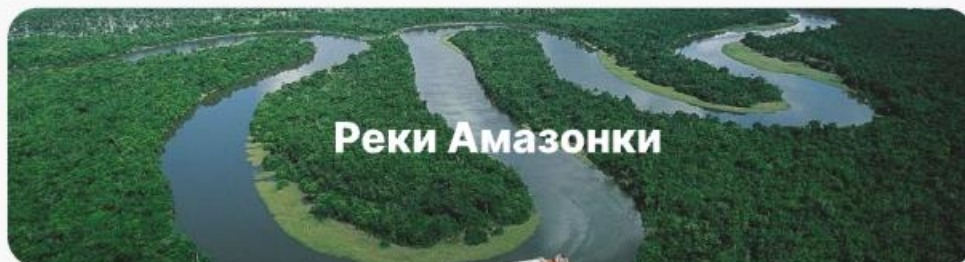


Рисунок 5 – Основная страница



Прохорова Мария
Николаевна

17.02.2026 в 16:30



Реки Амазонки

Описание статьи

Задумывались ли вы когда-то о величии природы изолированной Австралии?

Австралия стала изолированным архипелагом, давшим начало многим видам, которые затем распространились по всему земному шару. Примером такого вида может быть полевой крот, который устраивает ночью облаву на своих диких хищных соседей...



Оценить статью



Сайт разработчик правил курсовой работы, студентам ПИИ: 6-р-22-1 Софий

Контакт Telegram
2020, MIT LICENSE

Рисунок 6 – Основная страница

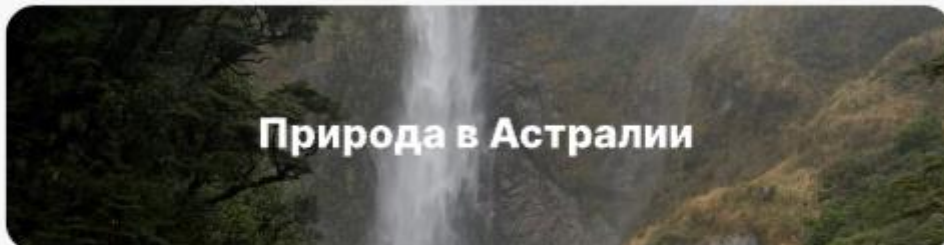
Создайте свою публикацию

Для этого нажмите на "Опубликовать" ниже, дальше следуйте инструкции на странице публикации, привлечите внимание потенциального читателя яркими заголовками

[Опубликовать](#)

 Природные Настроения
Николаева

17.02.2020 в 14:00



Природа в Австралии

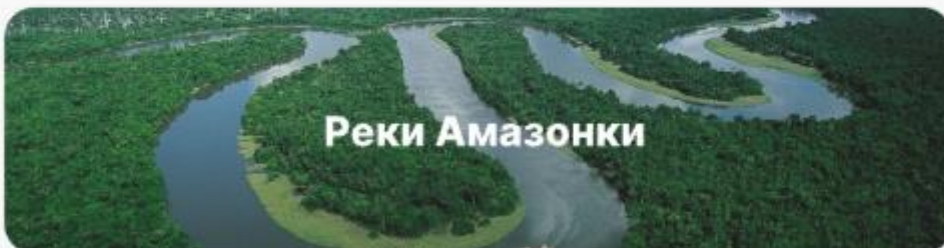
Подумайте, каковы будут последствия природы Австралии? Австралия стала известной как джунгли, давшие начало многим видам, которые затем распространились по всему миру (Шар). Примером такого вида может быть коала, который удерживает колючую оболочку на спине, а не на спине, как коала, которая...

★ ★ ★ ★

[Читать полную версию](#)

 Прозрачная Мария
Николаева

17.02.2020 в 10:40



Реки Амазонки

Подумайте, каковы будут последствия природы Австралии? Австралия стала известной как джунгли, давшие начало многим видам, которые затем распространились по всему миру (Шар). Примером такого вида может быть коала, который удерживает колючую оболочку на спине, а не на спине, как коала, которая...

★ ★ ★ ★

[Читать полную версию](#)

Рисунок 7 – Основная страница

Таблица 5 – Маршруты SPA приложения

Маршрут	Роль пользователя	Описание сценария и функционала
/	Любая	Корневой маршрут. Осуществляет редирект на /explore для ознакомления с контентом или на /auth, если требуется принудительная авторизация.
/auth	Любая	Страница авторизации. Пользователь вводит логин и пароль. При успехе сервер устанавливает HTTP[4]Only cookie с JWT, клиент перенаправляет пользователя в систему.
/register	Любая	Страница регистрации. Ввод ФИО, логина, пароля, пола и выбор роли. Данные валидируются на клиенте и сервере с использованием Zod[1].
/explore	Любая	Главная страница просмотра. Загружает список статей с поддержкой пагинации. Отображает карточки с заголовком, кратким описанием и именем автора.
/read/:id	Любая	Страница детального просмотра статьи. Запрашивает полные метаданные поста. Отображает текст, дату публикации и предоставляет ссылки на скачивание PDF и просмотр изображения из file storage.
/publish	Publisher	Страница создания публикации. Содержит форму с полями ввода и компонентами загрузки файлов. Отправляет multipart/form-data запрос на сервер для сохранения данных и медиа.

4.3. Оценка отзывчивости и пользовательского опыта (UX)

Разработанное приложение демонстрирует высокие показатели отзывчивости и удобства использования, что достигается за счет следующих архитектурных решений:

1. Архитектура Single Page Application (SPA). Отсутствие полных перезагрузок страницы при навигации обеспечивает плавный и быстрый пользовательский опыт, характерный для нативных десктопных приложений.

2. Асинхронное взаимодействие. Использование Axios с настроенными интерцепторами позволяет отображать индикаторы загрузки и мгновенно реагировать на ошибки (например, автоматический редирект на страницу входа при получении HTTP[4]-статуса 401).

3. Оптимизация рендеринга. Применение компонентного подхода React[2] и разделение логики по слоям Feature-Sliced Design минимизирует избыточные перерисовки интерфейса и упрощает поддержку кода.

4. Обратная связь с пользователем. Все критические действия (успешная публикация, ошибки валидации форм, удаление поста) сопровождаются модальными уведомлениями, что повышает предсказуемость и надежность системы.

5. Производительность сервера. Неблокирующая модель Node.js[7] в сочетании с асинхронными драйверами MongoDB[2] обеспечивает минимальную задержку при обработке запросов, включая операции ввода-вывода при загрузке файлов через Multer.

Заключение

В ходе выполнения курсового проекта была спроектирована и программно реализована полнофункциональная веб-платформа ScienceJournal для публикации и чтения научных статей.

Поставленная цель достигнута в полном объеме. Разработанное приложение успешно реализует клиент-серверную архитектуру с четким разделением ответственности: серверная часть на Node.js[7] и Express предоставляет защищенный REST API[5] с JWT-аутентификацией и файловым хранилищем, а клиентская часть на React[2] обеспечивает современный SPA-интерфейс, построенный по принципам Feature-Sliced Design.

Обоснованное отклонение от базового стека Django в пользу изоморфной JavaScript[2]-разработки (MERN-стек) позволило не только выполнить все требования методических указаний для траектории Б, но и заложить архитектурный фундамент для реализации расширенных функций траектории В, таких как real-time уведомления, кэширование и оптимистичные обновления интерфейса.

Система прошла этап функционального тестирования, подтвердив корректность работы ролевой модели, механизмов валидации данных и асинхронной загрузки медиафайлов. Полученный программный продукт обладает высокой степенью модульности, что гарантирует его легкую масштабируемость и возможность дальнейшей интеграции в производственную среду с использованием технологий контейнеризации, облачных хранилищ (CDN) и систем непрерывной интеграции (CI/CD).

Список использованной литературы

1. Библиотека валидации Zod : официальная документация [Электронный ресурс]. — Режим доступа: <https://zod.dev/> (дата обращения: 08.06.2026).
2. Васильев А. Н. Fullstack-разработка на JavaScript: React, Node.js, Express, MongoDB / А. Н. Васильев. — М. : ДМК Пресс, 2023. — 450 с.
3. Громов Г. Р. Основы разработки безопасных веб-приложений : учебное пособие / Г. Р. Громов. — СПб. : БХВ-Петербург, 2022. — 320 с.
4. Дакетт Д. Изучаем HTTP: руководство для веб-разработчиков / Д. Дакетт. — 2-е изд. — М. : O'Reilly, 2021. — 288 с.
5. Массе М. REST API. Разработка веб-сервисов / М. Массе. — М. : Вильямс, 2020. — 256 с.
6. Партыка Т. Л. Информационная безопасность : учебное пособие для вузов / Т. Л. Партыка, И. И. Попов. — М. : Форум, 2023. — 432 с.
7. Самойленко А. А. Node.js в действии / А. А. Самойленко. — 2-е изд. — М. : ДМК Пресс, 2022. — 416 с.
8. Соммервилл И. Инженерия программного обеспечения / И. Соммервилл. — 10-е изд. — М. : Вильямс, 2021. — 712 с.
9. Feature-Sliced Design : официальная документация [Электронный ресурс]. — Режим доступа: <https://feature-sliced.design/ru/docs> (дата обращения: 08.06.2026).
10. Чакрабартти С. MongoDB: полное руководство / С. Чакрабартти. — М. : Питер, 2023. — 384 с.

Ссылки на исходники

1. Github репозиторий проекта —
https://github.com/chainwarden771/efremova_science_journal
2. Макет проекта в Pixso —
https://pixso.net/app/design/8smpFWWrcj916pMEc40DhA?file_type=10&icon_type=1&page-id=0%3A1

Приложение А. JWT Middleware проверка серверной части

```
import { jwtVerify, decodeJwt, errors as JoseErros } from 'jose';

// =====
// Пользовательские импорты
// =====

import { findByLogin } from '../services/UserService.js';

// =====
// Окружение
// =====

const secret = new TextEncoder().encode(process.env.JWT_SIGN_KEY ||
'Sx02Ks');

export async function verifySession(req, res, next) {
  try {
    await jwtVerify(req.cookies.sessionKey, secret);
    const jwtPayload = await decodeJwt(req.cookies.sessionKey, secret);
    const dbUser = await findByLogin(jwtPayload.login);

    if (!dbUser) {
      throw new Error('Not found');
    }

    next();
  } catch (error) {
    res.clearCookie('sessionKey');
```



```
if (error instanceof JoseErrors.JWTExpired) {
  res.status(401).json({
    success: false,
    error: 'Session expired error',
  });
} else {
  res.status(401).json({
    success: false,
    error: 'Session error',
  });
}
console.error('Verify Session Error:', error);
}
```

Приложение Б. API слой на AJAX запросах

```
import { appInstance } from '..';

export const publishRequest = async ({ imageFile, pdfFile, title, description
}) => {
  const formData = new FormData();
  formData.append('image', imageFile);
  formData.append('pdf', pdfFile);

  const config = {
    headers: {
      'Content-Type': 'multipart/form-data',
    },
  };

  const metadata = JSON.stringify({
    post: {
      title,
      description,
    },
  });

  formData.append('metadata', metadata);
  const response = await appInstance.post('posts/publish', formData, config);
  return response.data;
};
```

Приложение В. Техническая записка проекта

Проект представляет собой SPA-платформу для публикации и чтения научных статей с серверной частью на Node.js + Express и клиентской частью на React.

Архитектура разделена на два основных слоя:

- client/ — React SPA (Vite)
- server/ — API + бизнес-логика + MongoDB + файловое хранилище

Таблица 1 – Стэк технологий

Категория	Технологии	Назначение
Frontend	React, React Router DOM	SPA интерфейс
HTTP	Axios	REST API взаимодействие
Backend	Node.js, Express	API сервер
Database	MongoDB (Mongoose/driver)	Хранение данных
Validation	Zod	Валидация входных данных
Auth	JWT + Cookies	Авторизация и сессии
Upload	Multer (multipart/form-data)	Загрузка файлов
Realtime	Socket.IO	(планируется/частично)
FS	fs/promises	Файловое хранилище
Code quality	ESLint, Prettier	Линтинг и форматирование
Modules	ESM (type: module)	Современная модульная система
API style	REST	CRUD операции
Pagination	offset/limit	Пагинация данных
Routing	Express Router + React Router	Маршрутизация
Params	URL params	Передача идентификаторов

1.1 Архитектура клиентской части (react spa)

Структура (FSD-like)

```
client/src/
├── api/          # HTTP слой (axios instances)
│   └── app/
│       ├── session/
│       └── posts/
```

```

|
| └─ components/      # UI компоненты (Header, Modal, Post)
| └─ features/        # бизнес-логика (auth, posts)
| └─ pages/           # страницы (Auth, Explore, Publish...)
| └─ shared/
|   └─ UI/            # переиспользуемые UI компоненты
|   └─ hooks/         # кастомные хуки
|   └─ context/       # глобальные контексты (Profile, Modal)
|   └─ utils/         # работа с storage/cookies
|   └─ consts/        # константы

```

1.2. Основные принципы

- UI отделён от бизнес-логики (features layer)
- Контексты используются для глобального состояния (user, modal)
- API слой изолирован через axios instances
- Логика авторизации вынесена в ProfileProvider

1.3 Архитектура серверной части

```

server/src/
└─ config/          # конфигурации (MongoDB)
└─ entities/        # модели (User, Post)
└─ middleware/      # auth, session verify
└─ routes/          # REST endpoints
  └─ session/
  └─ posts/
└─ services/        # бизнес-логика (DB abstraction)
└─ validation/      # Zod схемы

```

1.4 Api архитектура

POST /api/session/auth

POST /api/session/register

POST /api/session/logout

GET /api/session/check

1.5 Posts API

GET /api/posts/explore

GET /api/posts/:id

POST /api/posts/publish

DELETE /api/posts/:id

1.6 модель данных mongodb

User

```
{
  fullname: String, // max 45
  login: String,    // max 12
  password: String, // max 45
  gender: "m" | "f",
  role: "p" | "r",
  posts: [ObjectId]
}
```

Post

```
{
  title: String,
  description: String,
  postID: ObjectId (unique),
  author: ObjectId,
  publishDate: Number,
  imageFileName: String,
```

```
pdfFileName: String
}
```

1.7 файловое хранилище

```
server/file_storage/
├── posts/
│   ├── <postId>.png
│   └── <postId>.pdf
```

1.8 Принцип работы

- файлы связаны через postId
 - расширение сохраняется отдельно
 - доступ через express.static
- ```
app.use('/file_storage', express.static('./file_storage'));
```

## 1.9. Авторизация

Реализована по схеме JWT + Cookies:

1. Сервер создаёт JWT после успешной аутентификации
2. Токен отправляется через cookie 'sessionKey'
3. Middleware verifySession проверяет токен при каждом запросе
4. Профиль пользователя доступен через контекст ProfileProvider

## 1.10 валидация

Используется библиотека Zod для проверки данных на следующих этапах:

- регистрация пользователя
- авторизация
- создание постов

Пример использования:

```
registerSchema.parse(req.body);
```

## Приложение Г. Руководство пользователя

### 1. Введение

Система ScienceJournal представляет собой веб-платформу для публикации и чтения научных статей. В системе предусмотрены две основные роли пользователей: читатель и публицист. Читатель потребляет контент, просматривает и ищет статьи. Публицист обладает расширенными правами и может создавать и публиковать статьи.

### 2. Начало работы

#### 2.1. Регистрация

Для получения доступа к системе пользователю необходимо открыть страницу регистрации. В соответствующие поля вводятся имя пользователя, логин, пароль, выбирается пол и роль. После отправки формы сервер создает запись в базе данных, и пользователь автоматически авторизуется в системе.

#### 2.2. Вход в систему

Если у пользователя уже есть учетная запись, необходимо открыть страницу авторизации, ввести логин и пароль. При успешной проверке данных сервер выдает токен авторизации, который сохраняется в браузере, обеспечивая безопасность и непрерывность сессии.

### 3. Функционал читателя

#### 3.1. Просмотр списка статей

На главной странице отображается каталог доступных научных статей. Данные загружаются асинхронно с поддержкой пагинации, что позволяет удобно просматривать большое количество публикаций. Каждая статья представлена в виде карточки с заголовком и кратким описанием.

### 3.2. Чтение статьи

Для ознакомления с полным текстом публикации необходимо выбрать интересующую статью. Откроется детальная страница, содержащая текст статьи, информацию об авторе, дату публикации, а также прикрепленные медиафайлы.

## 4. Функционал публициста

### 4.1. Создание публикации

Пользователь с ролью публициста может создавать новые научные статьи. Для этого необходимо перейти на страницу публикации, заполнить форму, загрузить изображение для обложки и файл статьи. После отправки данные проходят валидацию, сохраняются в базе данных, а файлы помещаются в хранилище.

### 4.2. Управление своими публикациями

В разделе профиля отображается список всех статей, созданных данным пользователем. Отсюда можно перейти к чтению собственной статьи или удалить ее.

### 4.3. Удаление публикации

Для удаления статьи необходимо нажать соответствующую кнопку в списке своих публикаций. Сервер проверит права доступа, удалит запись из базы данных и очистит связанные файлы из хранилища.

## 5. Технические аспекты и безопасность

### 5.1. Обработка ошибок и сессии

Система автоматически контролирует срок действия сессии. При возникновении ошибок пользователь получает соответствующие уведомления через модальные окна. Ошибки валидации возвращают код 400, проблемы с авторизацией — 401, отсутствие ресурса — 404, внутренние сбои — 500.



## 5.2. Защита данных

Доступ к функциям создания и удаления публикаций строго ограничен ролью публициста. Все защищенные маршруты проверяются на наличие валидного токена. Загрузка файлов осуществляется с жесткой привязкой имен к идентификаторам постов, что исключает уязвимости и коллизии.

## 6. Перспективы развития

В будущих версиях системы планируется внедрение технологии реального времени для получения уведомлений о новых публикациях, а также добавление функций поиска, фильтрации и расширенного взаимодействия между пользователями.

### Приложение Д. Файловая структура проекта

#### Файловая структура проекта

```
├── client
│ ├── docs
│ │ ├── provider.md
│ │ ├── Методический материал.pdf
│ │ ├── МПП_Задание_Макет сайта.pdf
│ │ ├── Прототипирование сайта.pdf
│ │ └── Форматы данных.md
│ ├── eslint.config.js
│ ├── index.html
│ ├── package.json
│ ├── package-lock.json
│ ├── public
│ │ ├── assets
│ │ │ ├── example-doc.pdf
│ │ │ └── examples
```

```

| | | | └─ captcha-example.png
| | | | └─ post-image-example.jpg
| | | └─ fonts
| | | | └─ Bicubik-71qR.otf
| | | | └─ ReSquare-Bold.ttf
| | | | └─ ReSquare-Medium.ttf
| | | | └─ ReSquare-Regular.ttf
| | | └─ icons
| | | └─ rate-icon.svg
| | | └─ telegram.svg
| | └─ favicon.svg
| └─ readme.md
| └─ README.md
| └─ src
| | └─ api
| | | └─ app
| | | └─ index.js
| | | └─ posts
| | | | └─ explore.js
| | | | └─ publish.js
| | | | └─ read.js
| | | └─ session
| | | └─ auth.js
| | | └─ index.js
| | | └─ register.js
| | └─ App.css
| | └─ App.jsx
| | └─ assets
| | | └─ default-post-image.png
| | | └─ graduation-hat.png

```

```

| | | └─ hero.png
| | | └─ icons
| | | | └─ filter_alt.svg
| | | | └─ logo-ncfu-round-blue.svg
| | | | └─ profile-icon.svg
| | | | └─ rate-icon.svg
| | | | └─ react.svg
| | | | └─ telegram.svg
| | | | └─ vite.svg
| | | └─ pdf-icon.png
| | | └─ sceince-pic1.png
| | | └─ sceince-pic2.png
| | └─ components
| | | └─ Header
| | | | └─ Index.css
| | | | └─ Index.jsx
| | | └─ Modal
| | | | └─ Index.jsx
| | | | └─ Index.module.css
| | | | └─ Templates
| | | | | └─ Default.jsx
| | | | | └─ Default.module.css
| | | └─ Post
| | | | └─ Index.css
| | | | └─ Index.jsx
| | └─ features
| | | └─ posts
| | | | └─ usePost.js
| | | └─ session
| | | └─ useAuth.js

```

```

| | | └─ useRegister.js
| | └─ index.css
| | └─ main.jsx
| | └─ pages
| | | └─ Auth
| | | | └─ Index.css
| | | | └─ Index.jsx
| | | └─ Explore
| | | | └─ Index.css
| | | | └─ Index.jsx
| | | └─ Publish
| | | | └─ Index.css
| | | | └─ Index.jsx
| | | └─ Read
| | | | └─ Index.css
| | | | └─ Index.jsx
| | | └─ Register
| | | | └─ Index.css
| | | | └─ Index.jsx
| | | └─ Welcome
| | | | └─ Index.css
| | | | └─ Index.jsx
| | └─ shared
| | | └─ consts
| | | | └─ statusMessages.js
| | | └─ context
| | | | └─ ModalContext.jsx
| | | | └─ ModalProvider.jsx
| | | | └─ ProfileContext.jsx
| | | | └─ ProfileProvider.jsx

```

```

| | | └─ hooks
| | | | └─ useModalContext.js
| | | | └─ useProfileContext.js
| | | | └─ useProfile.js
| | | └─ UI
| | | | └─ Button
| | | | | └─ Index.jsx
| | | | | └─ Index.module.css
| | | | └─ Checkbox
| | | | | └─ Index.jsx
| | | | | └─ Index.module.css
| | | | └─ Input
| | | | | └─ Index.jsx
| | | | | └─ Index.module.css
| | | | └─ TextArea
| | | | | └─ Index.jsx
| | | | | └─ Index.module.css
| | | └─ utils
| | | └─ cookies.js
| | | └─ localStorage.js
| | | └─ sessionStorage.js
| └─ vite.config.js
└─ server
 └─ docs
 └─ examples.md
 └─ mongoDB.md
 └─ prompts.md
 └─ eslint.config.js
 └─ fileStorage
 └─ posts

```

```
| |─ 6a259f28ef552b7dcdcf611d.jpg
| |─ 6a259f28ef552b7dcdcf611d.pdf
| └─ index.js
| └─ package.json
| └─ package-lock.json
| └─ README.md
| └─ src
| └─ config
| └─ mongoDB.js
| └─ entities
| └─ Post.js
| └─ User.js
| └─ middleware
| └─ verifySession.js
| └─ routes
| └─ posts
| └─ explore.js
| └─ index.js
| └─ publish.js
| └─ read.js
| └─ session
| └─ auth.js
| └─ check.js
| └─ index.js
| └─ logout.js
| └─ register.js
| └─ services
| └─ mongoService.js
| └─ PostService.js
| └─ UserService.js
```

```
| └─ validation
| └─ zod
| └─ session.js
└─ static
└─ uploads
```

56 directories, 115 files

## ГЛОССАРИЙ ПРОЕКТА SCIENCE JOURNAL PLATFORM

### Общие термины

SPA (Single Page Application) — приложение, которое работает в браузере без полной перезагрузки страниц. Навигация осуществляется через React Router.

REST API — архитектурный стиль взаимодействия клиента и сервера через HTTP-запросы (GET, POST, PUT, DELETE).

CRUD — базовый набор операций над данными: создание (Create), чтение (Read), обновление (Update), удаление (Delete).

### Frontend (клиентская часть)

React — библиотека для построения пользовательских интерфейсов на основе компонентов.

React Router DOM — библиотека для маршрутизации внутри SPA, обеспечивающая переключение страниц без перезагрузки.

Axios — HTTP-клиент для отправки асинхронных запросов на сервер.

Context API — механизм React для организации глобального состояния (например, данные пользователя, модальные окна).

Feature (features/) — слой архитектуры, содержащий бизнес-логику (auth, posts и т.д.).

Shared layer — общий слой проекта, включающий UI-компоненты, хуки, утилиты и константы.

### Backend (серверная часть)

Node.js — среда выполнения JavaScript на сервере.

Express — фреймворк для построения HTTP API.

Router (Express Router) — механизм группировки API-маршрутов по логическому признаку (session, posts).

Middleware — функции, которые выполняются между получением запроса и отправкой ответа (например, проверка JWT).

### База данных



MongoDB — документо-ориентированная NoSQL база данных.

Mongoose / Mongo Driver — инструменты для работы с MongoDB из Node.js.

ObjectId — уникальный идентификатор документа в MongoDB.

Schema — структура документа в базе данных (например, User, Post).

Авторизация

JWT (JSON Web Token) — токен, используемый для идентификации пользователя без хранения сессий на сервере.

Cookie — механизм хранения данных в браузере, используется для хранения JWT.

Session — логическая сессия пользователя, основанная на связке JWT и cookie.

verifySession middleware — промежуточное ПО, проверяющее валидность JWT перед доступом к защищённым маршрутам.

Файлы и загрузка

Multer — middleware для обработки multipart/form-data (загрузка файлов).

Multipart/form-data — формат HTTP-запроса для передачи файлов и данных одновременно.

File Storage — локальное хранилище файлов на сервере (file\_storage/posts).

Express static — механизм отдачи файлов напрямую через HTTP.

Валидация

Zod — библиотека для схемной валидации данных на этапе выполнения (runtime validation).

Schema validation — проверка структуры данных перед сохранением в БД или обработкой.

Архитектура

FSD (Feature-Sliced Design) — подход к организации frontend-кода по слоям: app, pages, features, shared, components.

Service layer (backend) — слой, отделяющий бизнес-логику от маршрутов (routes).

Entity — сущность данных (User, Post), отражающая структуру БД.

Сеть и API

Base URL — базовый адрес API (например, `http://localhost:3000/api`).

Params (URL params) — параметры в URL, например `/posts/:id`.

Query params — параметры запроса, например `/posts?limit=10&page=2`

UI/UX

Modal — всплывающее окно поверх интерфейса.

Template (Modal Template) — предзаданный вариант содержимого модального окна.

Component — переиспользуемая часть UI (кнопка, инпут, пост).

Инструменты разработки

ESLint — инструмент проверки качества кода.

Prettier — инструмент форматирования кода.

ESM (ECMAScript Modules) — современная система импорта и экспорта модулей.

Дополнительные понятия

Pagination — разбиение данных на страницы для оптимизации загрузки.

Socket.IO — библиотека для real-time коммуникации между клиентом и сервером (абстракция над WebSocket).

Interceptor (Axios) — перехватчик запросов и ответов для обработки ошибок или добавления данных (например, JWT).

CORS — механизм контроля доступа между доменами (client и server).

dotenv — библиотека для загрузки переменных окружения из файла `.env`.